

Design & Analysis of Algorithms

Chapter 1: Introduction

By,

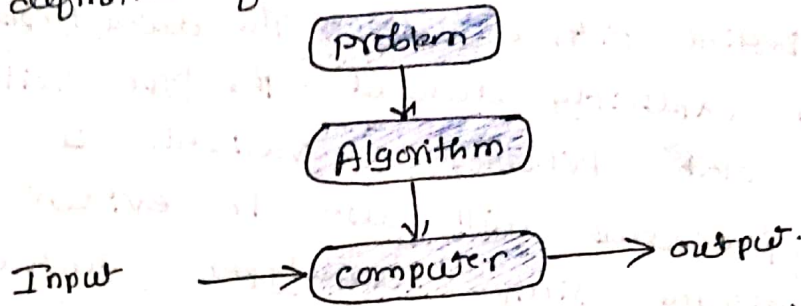
Prithviraj

Dept. of CSE

• What is an algorithm?

Definition: "Algorithm is a finite set of instructions for solving a given problem".

The definition of an algorithm can be represented as



Algorithms must satisfy the following criteria:

1) Input: Zero or more quantities are externally supplied.

2) Output: At least one quantity is produced.

3) Definiteness: Each instruction is clear and unambiguous.

4) Finiteness: If we trace out the instruction of an algorithm, then for all cases, the algorithm terminates after a finite number of steps.

5) Effectiveness: Every instruction must be very simple & should transform the given input to the desired output.

Algorithm Specifications:-

Algorithm is divided into 2 sections

↳ Algorithm heading

↳ Algorithm body

1. The heading consists of keyword Algorithm, name of the algorithm and parameter list.

Algorithm name $(P_1, P_2, \dots, P_n)$

2. Comments begin with // and continue until the end of the line

3. Blocks are indicated with matching braces $\{$ and $\}$. A compound statement can be represented as a block. The body of a procedure also forms a block. Statements are delimited by $;$

4. An identifier begins with a letter. The data type of variables are not explicitly declared. The type will be clear from the context. Whether a variable is global or local to a procedure will also be evident from the context. we assume simple datatypes such as int, float, char, boolean etc.

5. Assignment of value to variable is done using the assignment statement.

$\langle \text{variable} \rangle := \langle \text{expression} \rangle ;$

$a := 10 ;$

$z := x + y ;$

6. There are two boolean values 'true' & 'false'. In order to produce these values, the logical operator 'and', 'or' and 'not' & relational operators $<, \leq, >, \geq, =, \neq$ are provided.

7. Elements of multi-dimensional arrays are accessed using '[' and ']'. For example if A is a 2-dimensional array, the (i, j) th element of the array is denoted as $A[i, j]$. Array indices start with zero

8. Looping statements as follows: for, while and repeat until.

while :

while $\langle \text{Condition} \rangle$ do

```
{  
   $\langle \text{Statement 1} \rangle$   
   $\vdots$   
   $\langle \text{Statement n} \rangle$   
}
```

The General form of a for loop is:

for variable := value 1 to value 2 'step' step do

```
{  
   $\langle \text{Statement 1} \rangle$   
   $\vdots$   
   $\langle \text{Statement n} \rangle$   
}
```

Here value 1, value 2 and step are arithmetic expressions.

The repeat-until statement is constructed as,

```
repeat  
   $\langle \text{Statement 1} \rangle$   
   $\vdots$   
   $\langle \text{Statement n} \rangle$ 
```

until $\langle \text{Condition} \rangle$

The statements are executed as long as $\langle \text{Condition} \rangle$ is false.

9. A conditional statement has the following form

if $\langle \text{Condition} \rangle$ then $\langle \text{Statement} \rangle$

if $\langle \text{Condition} \rangle$ then $\langle \text{Statement} \rangle$ else $\langle \text{Statement 2} \rangle$

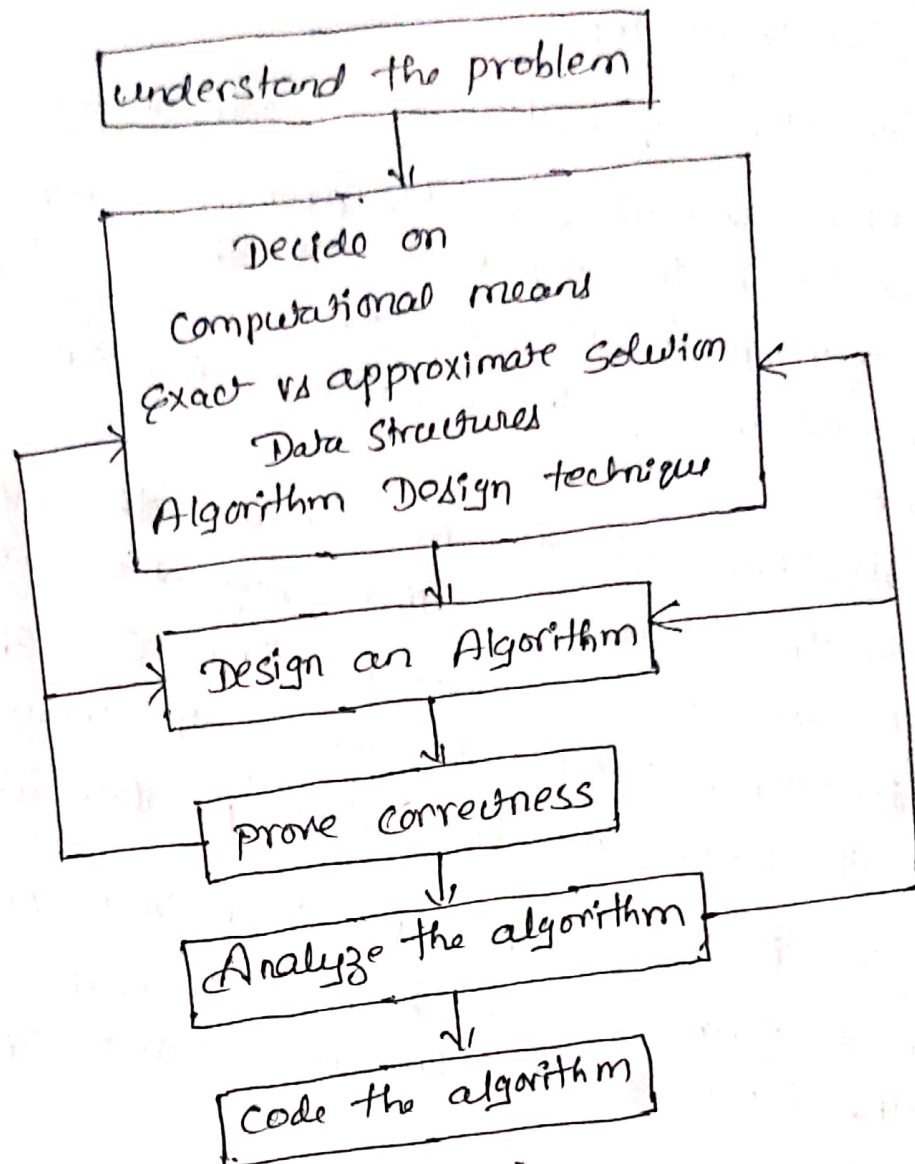
Here $\langle \text{Condition} \rangle$ is a boolean expression and $\langle \text{Statement} \rangle$,

$\langle \text{Statement 1} \rangle$ and $\langle \text{Statement 2} \rangle$ are arbitrary statements.

10. Input and output are done using the instructions 'read' and 'write'.

Fundamentals of Algorithmic Problem Solving:

Algorithm's design & Analysis process



understand the problem:

- First thing we need to do before designing an algorithm is to understand completely the given problem
- Read the problem's description carefully & ask questions if we have any doubts about the problem.
- An input to an algorithm specifies an instance of the problem the algorithm solves.
- It is important to specify exactly the range of instances the algorithm needs to handle.

→ Failing to do this, our algorithm may crash on some "boundary" values.

→ Correct algorithm is the one that works correctly for all legitimate inputs.

Ascertaining the capabilities of a computational device:

→ After understanding the problem completely, we need to ascertain the capabilities of the computational device the algorithm is intended for.

→ The essence of the architecture is captured by so called RAM.

→ Algorithms designed to execute the instructions one after the other are called 'sequential algorithms'.

→ In some newer computers RAM can operate parallelly.

→ Algorithms designed to execute the instructions concurrently at the same time are called 'parallel algorithms'.

Choosing between exact vs Approximate problem solving:

→ In the former case, an algorithm is called an exact algorithm; in the latter case, an algorithm is called approximation algorithm.

→ Approximation: There are some important problems that simply cannot be solved exactly for most of their instances eg: square root, non-linear equations, evaluating definite integrals.

→ Because of complexity of problems, the available algorithms for solving such problems can be unacceptably slow.

Deciding on appropriate Data structures:

→ Some algorithms do not demand any ingenuity in representing their inputs

Algorithms + Data Structures = Programs.

→ In new era of object oriented programming, data structures remain crucially important for both design and analysis of algorithms.

Algorithm Design Techniques:

→ It is a general approach for solving problems algorithmically that is applicable to a variety of problems from different areas of computing.

→ Learning these techniques important for the following reasons -

i) They provide guidance for designing algorithms for new problems.

ii) Algorithms are the cornerstone of Computer Science.

Methods of specifying an algorithm:

→ Once decided we need to specify it in some fashion.

There are 2 approaches:

Natural language: It has an obvious appeal. The inherent ambiguity of any natural language makes a succinct & clear description of algorithms surprisingly difficult.

A pseudocode: Is a mixture of natural language & programming language like constructs.

Proving an algorithm's correctness:

→ Once algorithm specified, we have to prove its correctness.

→ We have to prove that the algorithm yields required result for every legitimate input in a finite amount of time.

Analyzing an algorithm:

There are 2 types of algorithm efficiency:

Time efficiency & Space efficiency

Time efficiency - indicates how fast the algorithm runs.

Space efficiency - indicates how much extra memory the algorithm needs.

- Another desirable characteristic of an algorithm is simplicity, next desirable characteristic of an algorithm is generality. - 2 issues: generality of the problem the algorithm solves & range of inputs it accepts
- If algorithm's efficiency is not satisfied, simplicity, or generality we must return to the drawing board & redesign the algorithm.

coding an Algorithm:

- Programming an algorithm presents both peril and an opportunity. The peril lies in the possibility of making the transition from algorithm to program either incorrectly or very inefficiently
- we assume that inputs to algorithms fall within their specified ranges & hence require no verification.
- A working program provides an additional opportunity in allowing an empirical analysis of the underlying algorithm.

Recall greatest common divisor of two nonnegative integers m & n , denoted as $\text{gcd}(m, n)$
→ Euclid's algorithm is based on applying repeatedly the equality.

Euclid's algorithm for computing $\text{gcd}(m, n)$
Step 1: If $n=0$, return the value of ' m ' as the answer and stop; otherwise go to Step 2
Step 2: Divide m by n & assign the value of the remainder to r
Step 3: Assign the value of n to m & value of r to n . Go to Step 1.

Algorithm Euclid(m, n)
1 Computed $\text{gcd}(m, n)$ by Euclid's algorithm
2 Input: Two non negative integers m & n
3 Output: GCD of m & n

```
while  $n \neq 0$  do  
     $r \leftarrow m \bmod n$   
     $m \leftarrow n$   
     $n \leftarrow r$   
return  $m$ 
```

→ In this second number of the pair gets smaller with each iteration & it cannot become negative. Hence value of 2nd number in the pair eventually becomes 0, & the algorithm stops.

Consecutive integer checking algorithm.

Step 1: Assign the value of $\min(m, n)$ to t
Step 2: Divide m by t . If the remainder of this division is 0 go to Step 3 else go to Step 4.
Step 3: divide n by t . If the remainder of this division is zero return value of ' t ' as answer and stop; else go to Step 4

Step 4: Decrease the value of 'c' by 1; go to Step 2
Third procedure for finding the gcd is familiar from middle school.

Middle-school procedure

Step 1: Find the prime factors of m

Step 2: Find the prime factors of n

Step 3: Identify all common factors in the two prime expansions found in Step 1 and Step 2.

Step 4: Compute the product of all the common factors and return it as the gcd

Ex: $\text{gcd}(60, 24)$

$$60 = \underline{2} \cdot \underline{2} \cdot \underline{3} \cdot 5$$

$$24 = \underline{2} \cdot \underline{2} \cdot \underline{2} \cdot \underline{3}$$

$$\text{gcd}(60, 24) = 2 * 2 * 3 = \underline{\underline{12}}$$

11. There is only one type of procedure : "Algorithm".

The algorithm body has one or more statements enclosed within { and } braces.

EX:- 1. Following algorithm finds & returns maximum of 'n' given numbers.

1. Algorithm max(A, n)
2. // A is an array of size n
3. {
4. Result := A[1];
5. for i = 2 to n do
6. if $A[i] > \text{Result}$ then Result := A[i];
7. return Result;
8. }

2) Algorithm to count the sum of 'n' natural numbers.

1. Algorithm Sum(1, n)
2. // input the numbers from 1 to n
3. {
4. result := 0;
5. for i = 1 to n do
6. result = result + i;
7. return result;
8. }

3) Algorithm to find the factorial of a number

1. Algorithm fact(n)
2. // input- any valid number
3. {
4. factorial := 1;
5. for i = 1 to n do
6. factorial := factorial * i;

7. return factorial

8. }

Analysis Framework:-

Analysis framework describe a general framework for analyzing the efficiency of an algorithm. The efficiency of an algorithm depends on two factors.

1) Space efficiency

2) Time efficiency.

→ The space efficiency of an algorithm is the amount of memory required to run the program completely and efficiently.

→ Space complexity of an algorithm depends on following factors

↳ Program space - Space required to store machine program

↳ Data space - space required to store constants, variables etc.

↳ Stack Space.- space required to store return address along with parameter.

→ Time efficiency of an algorithm is measured purely on how fast a given algorithm is executed. Since the efficiency of an algorithm is measured using time.

Components that affect time efficiency

- Speed of the computer
- Choice of the programming language
- Compiler used.
- Choice of the algorithm.
- Number of inputs / outputs

Orders of Growth:-

Definition: Measuring the performance of an algorithm in relation with the input size 'n' is called "order of growth".

• Best-case Analysis: If an algorithm takes minimum amount of time to run to completion for a specific set of input then it is called 'best case' time complexity.
EX: while searching a particular element in linear search if we get the desired element at first place itself then it is called best case.

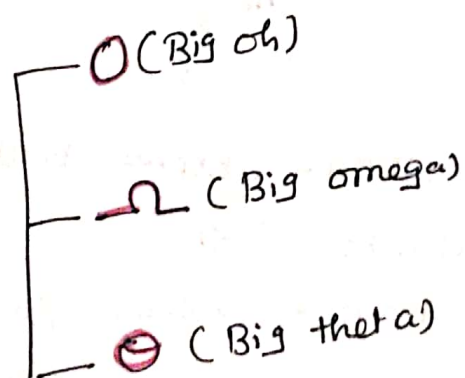
Worst case Analysis: If an algorithm takes maximum amount of time to run to completion of specific set of input then it is called 'worst case' time complexity.
EX: while searching a particular element in linear search if we get the desired element at last place then it is called worst case.

Average case Analysis: The time complexity that we get for certain set of inputs is as a average same. Such a time complexity called average time complexity.

Asymptotic Notations: [mathematical way of representing the time complexity]

Definition: The value of the function may increase or decrease as the value of 'n' increases. Based on the order of growth of n the behaviour of the function varies. Asymptotic notations are the notations using which two algorithms can be compared with respect to efficiency based on order of growth of an algorithm's basic operation.

Asymptotic notations



Big O (oh): (Upper Bound)

Informal definition of asymptotic notation: Big O

Assume 'n' indicates the size of input and $f(n)$ is a function, informally $O(g(n))$ is defined as set of functions with a small or same order of growth as $g(n)$.

Eg: $n \in O(n^3)$

$$n^2 \in O(n^3)$$

$$2n^3 \in O(n^3)$$

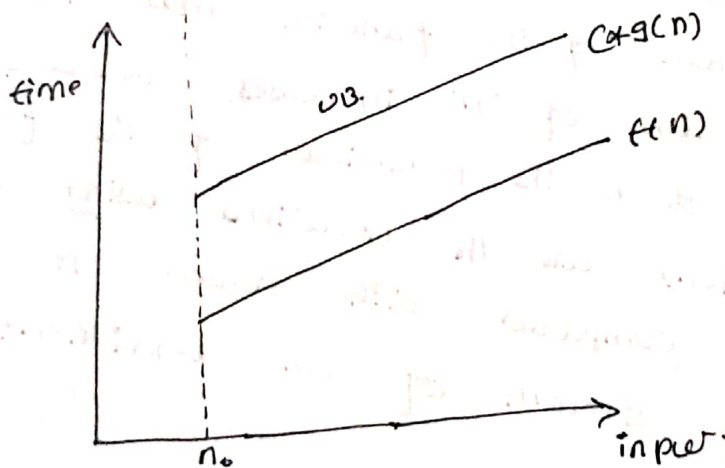
$$0.1n^4 \notin O(n^3)$$

$$n^5 + 2n + 5 \notin O(n^3)$$

Formal definition: Big O

Let $t(n)$ be the time efficiency of an algorithm. A function $t(n)$ is said to be $O(g(n))$ denoted by $t(n) \in O(g(n))$ such that there exists a positive constant c and positive integer n_0 satisfying the constraint $t(n) \leq c * g(n)$ for all $n \geq n_0$.

So if we draw the graph $t(n)$ and $c * g(n)$ versus n , the graph of the function $t(n)$ lies below the graph of $c * g(n)$ for sufficiently large values of 'n' as



Problem:

1) Let $t(n) = 100n + 5$. Express $t(n)$ using Big O

P.T
 $100n + 5 \in O(n)$

$$\rightarrow t(n) = 100n + 5 \quad \text{--- (1)}$$

we have $t(n) \leq c * g(n)$ for all $n \geq n_0$

Replace '5' with 'n' in equation (1)

So that next higher order term is obtained.

we will get $100n + n$ and call it as $g(n)$

$$c * g(n) = 100n + n$$

$$c * g(n) = 101n$$

$$t(n) \leq c * g(n) \quad \forall n \geq n_0$$

$$100n + 5 \leq 101n$$

$$\text{if } n=1, \quad 105 \not\leq 101$$

$$n=2, \quad 205 \not\leq 202$$

$$n=3, \quad 305 \not\leq 303$$

$$n=4, \quad 405 \not\leq 404$$

$$\text{For } n=5, \quad 505 \leq 505 \quad \text{so,}$$

$$100n + 5 \leq 101n \quad \text{for } n \geq 5$$

It is clear from the above relation that $c=101, g(n)=n, n_0=5$.

$$\text{Hence } t(n) \in O(g(n)), \quad t(n) \in O(n)$$

2) Let $f(n) = 10n^3 + 8$. Express $f(n)$ using Big O

solⁿ: Given that $f(n) = 10n^3 + 8$

Replace 8 with n^3 (to obtain ^{next} highest order term). we get

$$10n^3 + n^3$$

$$\text{i.e. } c * g(n) = 10n^3 + n^3 = 11n^3$$

$$10n^3 + 8 \leq 11n^3$$

$$\text{for } n=1$$

$$10 + 8 \not\leq 11$$

$$n=2,$$

$$10 * 8 + 8 \leq 88 \Rightarrow 88 \leq 88, \quad n \geq 2$$

$$\text{i.e. } 10n^3 + 8 \leq 11n^3 \quad \text{for } n \geq 2$$

that means $c=11, g(n)=n^3 \quad \& \quad n_0=2$

$$\boxed{f(n) \in O(g(n))}$$

$$\text{i.e. } \boxed{f(n) \in O(n^3)}$$

3). Let $t(n) = 6 \times 2^n + n^2$ express $t(n)$ using Big O(O)

$$\Rightarrow t(n) = 6 \times 2^n + n^2 \quad \text{--- (1)}$$

From the definition, we have $t(n) \leq c * g(n)$ for all $n \geq n_0$

Replace 6×2^n with n^2 in (1)

(5)

we get $n^2 + n^2$ call it as $g(n)$

$$C * g(n) = n^2 + n^2$$

$$C * g(n) = 2n^2$$

$$t(n) \leq C * g(n)$$

$$6 * 2^n + n^2 \leq 2n^2 \quad \forall n \geq n_0$$

or

Replace n^2 with 2^n we get $6 * 2^n + 2^n$

$$C * g(n) = 6 * 2^n + 2^n$$

$$= 7 * 2^n$$

$$t(n) \leq C * g(n)$$

$$6 * 2^n + n^2 \leq 7 * 2^n$$

$$6 + 1 \leq 7$$

$$7 \leq 7$$

i.e. $6 * 2^n + n^2 \leq 7 * 2^n$ for all $n \geq 4$

$$C = 7, g(n) = 2^n, n_0 = 0$$

$$f(n) \in O(g(n)) \quad \text{i.e. } f(n) \in O(2^n)$$

$$\text{4) P.T. } 3n^3 + 2n^2 = O(n^3)$$

$$\Rightarrow t(n) \leq C * g(n)$$

$$t(n) \in O(g(n))$$

$$\text{Assume } f(n) = 3n^3 + 2n^2$$

$$g(n) = n^3$$

$$3n^3 + 2n^2 \leq n^3$$

$$n \geq 1, 5 \leq 1$$

$$n \geq 2, C = 4$$

$$3(2)^3 + 2(2)^2 \leq 4 \cdot 2^3$$

$$24 + 8 \leq 4 \cdot 8$$

$$32 \leq 32$$

Replace $2n^2$ with n^3
[to obtain next higher term]

$$\text{P.T. } 10n^2 + 4n + 2 \in O(n^2)$$

∴ Hence

$$3n^3 + 2n^2 = O(n^3)$$

$$6) \text{ P.T } 2n^2 + 7 = O(n^2)$$

→ From the definition we know that

$$f(n) \leq C * g(n) \quad \forall n > n_0$$

$$f(n) = 2n^2 + 7$$

Replace 2 with n^2 . then we get

$$2n^2 + n^2 = 3n^2$$

$$2n^2 + 7 \leq 3n^2$$

$$n=0, 7 \not\leq 0 \quad n=1, 9 \not\leq 3$$

$$n=2, 15 \not\leq 12$$

$$n=3, 25 \leq 27$$

$$2n^2 + 7 < 3n^2 \quad \text{for } n \geq 3$$

Hence $f(n) \in O(g(n))$ i.e. $f(n) \in O(n^2)$

$$C=3, n_0=3 \text{ \& } g(n)=n^2$$

Big Omega (Ω) : (Lower Bound)

Informal definition - Assuming 'n' indicates the size of the input and $g(n)$ is a function, informally $\Omega(g(n))$ is defined as set of functions with a large or same order of growth as $g(n)$.

Let $g(n) = n^3$ then

$$n^3 \in g(n^3)$$

$$n^4 \in g(n^3)$$

$$0.001n^4 \in g(n^3)$$

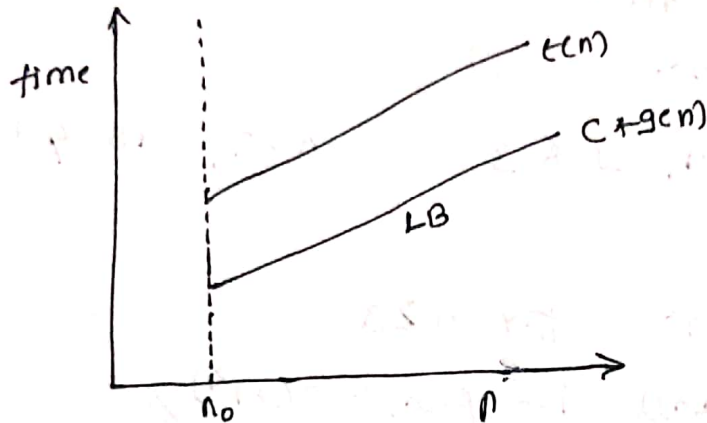
$$2n + 3 \notin g(n^3)$$

$$n^2 + n + 4 \notin g(n^3)$$

Formal definition:

Let $t(n)$ be the time complexity of an algorithm. The function $t(n)$ is said to be $\Omega(g(n))$ denoted as $t(n) \in \Omega(g(n))$ or $t(n) = \Omega(g(n))$ such that there exists a positive constant 'c' & non-negative integer n_0 satisfy the constraint $t(n) \geq c * g(n)$ for all $n \geq n_0$.

If we draw the graph $t(n)$ and $c * g(n)$ versus n , the graph of $t(n)$ lies above the graph of $g(n)$.



Problem:-

Let $t(n) = 100n + 5$, express $t(n)$ using big omega (Ω)
→ we have $t(n) \geq c * g(n) \quad \forall n \geq n_0$

Given: $t(n) = 100n + 5$

Replace '5' with 0 (such that lower term is obtained)
After we get $100n + 0$ & call it as $c * g(n)$

$$c * g(n) = 100n$$

$$100n + 5 \geq 100n$$

$$n \geq 0, \quad 0 + 5 \geq 0$$

$$5 \geq 0$$

$$100n + 5 \geq 100n \quad \text{for all } n \geq 0$$

Hence $t(n) \geq c * g(n)$

$$t(n) \in \Omega(g(n)) \quad \text{i.e. } \boxed{t(n) \in \Omega(n)}$$

2. Let $t(n) = 10n^3 + 5$. Express $t(n)$ using big omega.

→ we have $t(n) \geq c \cdot g(n) \quad \forall n \geq n_0$

$$\text{Given } t(n) = 10n^3 + 5$$

Replace 5 with 0 (to obtain lower order term)

After replacing we get, $10n^3 + 0$ & we call it as $c \cdot g(n)$.

$$c \cdot g(n) = 10n^3$$

$$t(n) \geq c \cdot g(n)$$

$$10n^3 + 5 \geq 10n^3 \quad \forall n \geq 0$$

It's clear from the equation that $c = 10, n_0 = 0, g(n) = n^3$.

hence $t(n) \in \Omega(g(n))$ i.e. $t(n) \in \Omega(n^3)$

3) Let $t(n) = 6 \times 2^n + n^2$. Express $t(n)$ using Big omega.

→ we have $t(n) \geq c \cdot g(n) \quad \forall n \geq n_0$

$$\text{Given } t(n) = 6 \times 2^n + n^2$$

Replace n^2 with 0, we get $6 \times 2^n + 0$ we call this

as $c \cdot g(n)$

$$c \cdot g(n) = 6 \times 2^n$$

$$t(n) \geq c \cdot g(n) \quad \forall n \geq n_0$$

$$6 \times 2^n + n^2 \geq 6 \times 2^n \quad \forall n \geq 0$$

$\therefore t(n) \in \Omega(g(n))$ i.e. $t(n) \in \Omega(2^n)$

4) Show that $5x^2 \notin \Omega(x^3)$

→ we have $t(n) \geq c \cdot g(n) \quad \forall n \geq n_0$

$$5x^2 \geq x^3 \quad \forall x \geq 6$$

$$x=1 \quad 5 \geq 3, \quad x=2 \quad 20 \geq 8, \quad x=3 \quad 45 \geq 27 \dots$$

$$x=6, \quad 180 \not\geq 216$$

Hence $5x^2 \notin \Omega(x^3)$

6) S.T $5x^2 \in O(x^3)$

→ we have $t(n) \leq c * g(n) \quad \forall n \geq n_0$

$$5x^2 \leq x^3$$

$$x=1, 5 \leq 1; \quad x=2, 20 \leq 8; \quad x=3, 45 \leq 27; \quad x=4,$$

$$x=5, 125 \leq 125 \checkmark$$

$$5x^2 \leq x^3 \quad \forall x \geq 5$$

$$\therefore \boxed{5x^2 \in O(x^3)}$$

6) P.T $3n^3 + 2n^2 \in O(n^3)$

→ we have $t(n) = 3n^3 + 2n^2$

we know that $t(n) \leq c * g(n) \quad \forall n \geq n_0$

Replace $2n^2$ with n^3 we get $3n^3 + n^3 \leq$ call it as $c * g(n)$

$$c * g(n) = 4n^3$$

$$t(n) \leq c * g(n)$$

$$3n^3 + 2n^2 \leq 4n^3$$

$$n=1, \quad 5 \nless 4$$

$$n=2, \quad 32 \leq 32$$

$$n=3, \quad 99 \leq 108 \quad \text{L.H.S.}$$

$$3n^3 + 2n^2 \leq 4n^3, \quad \forall n \geq 2$$

$$\therefore 3n^3 + 2n^2 \in O(n^3) \quad \text{i.e. } t(n) \in O(n^3)$$

7) P.T $3^n \notin O(2^n)$

→ we know that $t(n) \leq c * g(n) \quad \forall n \geq n_0$

$$t(n) \leq c * g(n)$$

$$3^n \leq 2^n$$

$$\text{But } 3^n \nless 2^n \quad \forall n \geq 1$$

$$\text{Hence } \boxed{3^n \notin O(2^n)}$$

Big Theta (Θ) notation:

Assuming 'n' indicates the size of input & $g(n)$ is a function, informally $\Theta(g(n))$ is defined as set of functions that have same order of growth as $g(n)$.

Eg: $g(n) = n^3$

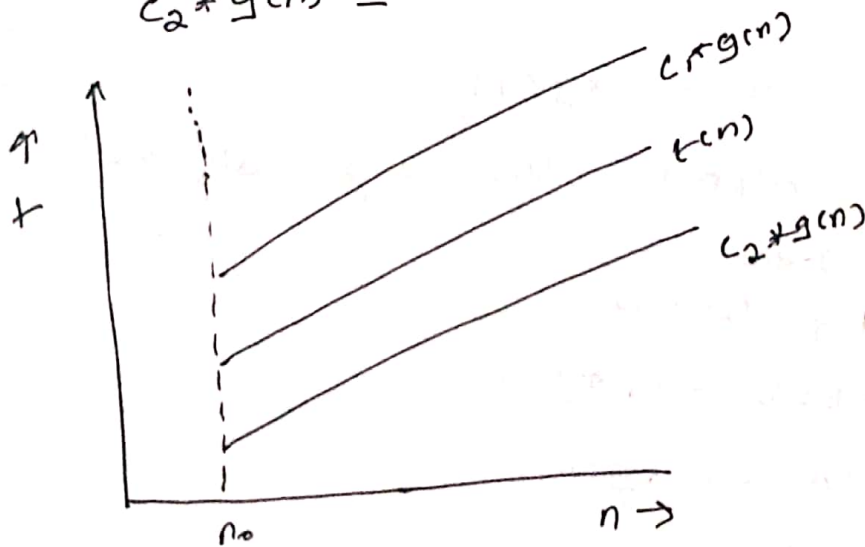
$$n^3 \in \Theta(n^3)$$

$$n^4 \in \Theta(n^3)$$

$$n^5 + n + 3 \in \Theta(n^3)$$

Definition: A function $t(n)$ is said to be in $\Theta(g(n))$ denoted by $t(n) \in \Theta(g(n))$, if $t(n)$ is bounded both above and below by some positive constant multiples of $g(n)$ for all larger 'n' i.e. if there exist some positive constant c_1 and c_2 and some non-negative integer n_0 such that

$$c_2 * g(n) \leq t(n) \leq c_1 * g(n) \quad \forall n \geq n_0$$



1) Let $t(n) = 100n + 5$, express $t(n)$ using Big theta.

→ we have $t(n) = 100n + 5$

we have constraint of big theta

$$c_2 * g(n) \leq t(n) \leq c_1 * g(n)$$

Let us first consider

$$t(n) \leq c_1 * g(n)$$

It is the constraint of Big Oh.

(R)

Let us replace 5 by n we get $100n + n = 101n$
we call it as $C_1 * g(n)$

$$C_1 * g(n) = 101n$$

$$\therefore t(n) \leq C_1 * g(n)$$

$$100n + 5 \leq 101n \quad \text{--- (1)} \quad \forall n \geq 5$$

where $C = 101$, $g(n) = n$, $n_0 = 5$

Next let us consider $t(n) \geq C_2 * g(n)$ which is the constraint of big omega

$$C_2 * g(n) = 100n \quad \text{By replacing 5 with 0}$$

$$t(n) \geq C_2 * g(n)$$

$$\therefore 100n + 5 \geq 100n \quad \forall n \geq 0 \quad \text{--- (2)}$$

From (1) & (2)

$$100n \leq 100n + 5 \leq 101n \quad \forall n \geq 5$$

$$\therefore C_2 * g(n) \leq t(n) \leq C_1 * g(n)$$

3) Let $t(n) = 10n^3 + 5$. Express $t(n)$ using Big Theta

→ we have $t(n) = 10n^3 + 5$

The constraint of big theta is

$$C_2 * g(n) \leq t(n) \leq C_1 * g(n)$$

Let us consider $t(n) \leq C_1 * g(n)$

$$10n^3 + 5$$

replace 5 by n^3 we get $10n^3 + n^3 = 11n^3$ we call

it as $C_1 * g(n)$

$$C_1 * g(n) = 11n^3$$

$$10n^3 + 5 \leq 11n^3 \quad \forall n \geq 2 \quad \text{--- (1)}$$

Next consider $t(n) \geq C_2 * g(n)$

$10n^3 + 5$ Replace 5 by 0 we get $10n^3$ &

we call it as $C_2 * g(n)$

$$c_2 * g(n) = 10n^3$$

$$10n^3 + 5 \geq 10n^3 \quad \forall n \geq 0 \quad \text{--- (2)}$$

From (1) & (2).

$$10n^3 \leq 10n^3 + 5 \leq 11n^3 \quad \forall n \geq 2.$$

$$\therefore c_2 * g(n) \leq f(n) \leq c_1 * g(n)$$

Theorem:- If $f_1(n) \in O(g_1(n))$ and $f_2(n) \in O(g_2(n))$ then $f_1(n) + f_2(n) \in O(\max\{g_1(n), g_2(n)\})$

Proof: By definition, we know that the function $f(n)$ is said to be in $O(g(n))$ denoted by $f(n) \in O(g(n))$ such that there exist a positive constant C & positive integer n_0 satisfying the constraint $f(n) \leq C * g(n) \quad \forall n \geq n_0$

It is given that $f_1(n) \in O(g_1(n))$ so by definition there exist a relation $f_1(n) \leq c_1 * g_1(n) \quad \forall n \geq n_1 \quad \text{--- (1)}$

It is also given that $f_2(n) \in O(g_2(n))$ so by definition there exist a relation $f_2(n) \leq c_2 * g_2(n) \quad \forall n \geq n_2 \quad \text{--- (2)}$

Let us assume that

$$c_3 = \max\{c_1, c_2\}$$

$$n_3 = \max\{n_1, n_2\}$$

By adding (1) & (2) we get

$$\begin{aligned} f_1(n) + f_2(n) &\leq c_1 * g_1(n) + c_2 * g_2(n) \\ &\leq c_3 * g_1(n) + c_3 * g_2(n) \quad \forall n \geq n_3 \\ &\leq c_3 \{g_1(n) + g_2(n)\} \end{aligned}$$

[From mathematical statement if $a_1 \leq b_1$ & $a_2 \leq b_2$ then $a_1 + a_2 \leq 2 \max(b_1, b_2)$]

$$\therefore t_1(n) + t_2(n) \leq c_3 \{ 2 \max \{ g_1(n), g_2(n) \} \}$$

$$\therefore \boxed{t_1(n) + t_2(n) \in O(\max(g_1(n), g_2(n)))}$$

where $2c_3$ is a constant

Order of Growth using Limits:

The order of growth can be obtained by comparing the limit of the ratio of the two functions as shown below.

$$\lim_{n \rightarrow \infty} \frac{t(n)}{g(n)} = \begin{cases} 0 & \text{implies that } t(n) \text{ has a smaller order of growth than } g(n) \text{ i.e. } t(n) \in O(g(n)) \\ c > 0 & \text{implies that } t(n) \text{ has a same order of growth as } g(n) \text{ i.e. } t(n) \in \Theta(g(n)) \\ \infty & \text{implies that } t(n) \text{ has a larger order of growth than } g(n) \text{ i.e. } t(n) \in \Omega(g(n)) \end{cases}$$

Example 1: $\rightarrow$ Compare the order of growth of $\frac{1}{2}n(n-1)$ and n^2 using limits.

$\rightarrow$ According to Stirling's formula

$$n! = \sqrt{2\pi n} \left(\frac{n}{e}\right)^n \quad \text{for large values of 'n'}$$

[Not used for this problem]

$$\rightarrow \text{Given } t(n) = \frac{1}{2}n(n-1) \quad \& \quad g(n) = n^2$$

$$\begin{aligned} \lim_{n \rightarrow \infty} \frac{t(n)}{g(n)} \\ = \lim_{n \rightarrow \infty} \frac{\frac{1}{2}n(n-1)}{n^2} \end{aligned}$$

$$= \frac{1}{2} \lim_{n \rightarrow \infty} \frac{n-1}{n} = \frac{1}{2} \lim_{n \rightarrow \infty} \left(1 - \frac{1}{n}\right)$$

$$= \frac{1}{2} \left(1 - \frac{1}{\infty}\right) \quad \left[\text{But } \frac{1}{\infty} = 0 \right]$$

$$= \frac{1}{2} \quad \text{Hence } \boxed{\frac{1}{2}n(n-1) \in \Theta(g(n^2))}$$

Q2) Compare the order of growth of $\log_2 n$ and $\sqrt{n}$ using limits.

→ Given $t(n) = \log_2 n$ $g(n) = \sqrt{n}$

$$\lim_{n \rightarrow \infty} \frac{t(n)}{g(n)} = \lim_{n \rightarrow \infty} \frac{\log_2 n}{\sqrt{n}}$$

According to mathematical statement $\log_a n$ can be written as $\frac{\log_e n}{\log_e a}$ then $\left[\because \log_a b = \frac{\log_e b}{\log_e a} \right]$

$$\lim_{n \rightarrow \infty} \frac{\frac{\log_e n}{\log_e 2}}{\sqrt{n}} = \lim_{n \rightarrow \infty} \frac{1/\log_e 2 (\log_e n)}{\sqrt{n}}$$

$$\lim_{n \rightarrow \infty} \frac{\log_e (\log_e n)}{\sqrt{n}} \quad \left[\because \frac{1}{\log_n m} = \log_m n \right]$$

$$= \log_2 e \lim_{n \rightarrow \infty} \frac{\log_e n}{\sqrt{n}}$$

According to L'Hospital's rule indeterminate forms can be solved by applying the differentiation

$$\lim_{n \rightarrow \infty} \frac{t(n)}{g(n)} = \lim_{n \rightarrow \infty} \frac{t'(n)}{g'(n)} \quad \text{i.e.} \quad \lim_{n \rightarrow \infty} \frac{(\log_e n)'}{(\sqrt{n})'}$$

$$\log_e n = \frac{d}{dn} (\log_e n) = 1/n \quad - (1)$$

$$\sqrt{n} = \frac{d}{dn} (\sqrt{n}) = \frac{d}{dn} (n)^{1/2}$$

$$= 1/2 n^{1/2-1}$$

$$\left[\because x^n = nx^{n-1} \right]$$

$$= 1/2 n^{-1/2} = \frac{1}{2 \cdot n^{1/2}} = \frac{1}{2\sqrt{n}} \quad - (2)$$

By taking eqn (1) and (2)

$$\lim_{n \rightarrow \infty} \frac{t(n)}{g(n)} = \log_2 e \left[\lim_{n \rightarrow \infty} \frac{1/n}{\frac{1}{2\sqrt{n}}} \right]$$

$$= \log_2 e \left(\lim_{n \rightarrow \infty} \frac{2\sqrt{n}}{n} \right)$$

$$= 2 \cdot \log_2 e \left(\lim_{n \rightarrow \infty} \frac{\sqrt{n}}{n} \right) = 2 \cdot \log_2 e \left(\lim_{n \rightarrow \infty} \frac{(n)^{1/2}}{n} \right)$$

$$= 2 \cdot \log_2 e \left(\lim_{n \rightarrow \infty} \frac{1}{n^{1/2}} \right) = 2 \log_2 e \left(\lim_{n \rightarrow \infty} \frac{1}{n^{1/2}} \right)$$

$$= 2 \cdot \log_2 e \left(\lim_{n \rightarrow \infty} \frac{1}{\sqrt{n}} \right) = 2 \cdot \log_2 e \times \frac{1}{\infty}$$

Hence $t(n) = \log_2 n$ has a smaller growth of $g(n) = \sqrt{n}$

i.e. $\boxed{\log_e n \in O(\sqrt{n})}$

3) Compare the order of growth of $n!$ and 2^n .

→ Given $t(n) = n!$ $g(n) = 2^n$
According to Stirling's formula $n! = \sqrt{2\pi n} (n/e)^n$

$$\lim_{n \rightarrow \infty} \frac{t(n)}{g(n)} = \lim_{n \rightarrow \infty} \frac{n!}{2^n}$$

$$= \lim_{n \rightarrow \infty} \frac{\sqrt{2\pi n} \cdot (n/e)^n}{2^n}$$

$$= \lim_{n \rightarrow \infty} \frac{\sqrt{2\pi n} \cdot \frac{n^n}{e^n}}{2^n} = \lim_{n \rightarrow \infty} \sqrt{2\pi n} \cdot \frac{n^n}{2^n \cdot e^n}$$

$$= \lim_{n \rightarrow \infty} \sqrt{2\pi n} \cdot \left(\frac{n}{2e} \right)^n$$

$$= \infty$$

Hence $t(n) = \boxed{n! \in \Omega(2^n)}$

4) Show that $5x^2$ is $O(x^3)$ using limits

$$\rightarrow t(n) = 5x^2 \quad g(x) = x^3$$

$$\lim_{x \rightarrow \infty} \frac{t(n)}{g(n)} = \lim_{x \rightarrow \infty} \frac{5x^2}{x^3}$$

$$= 5 \cdot \lim_{x \rightarrow \infty} \frac{1}{x}$$

$$= 5 \times \frac{1}{\infty} = 5 \times 0 = 0 //$$

$$\therefore t(n) \in O(g(n))$$

$$\text{i.e. } \boxed{5x^2 \in O(x^3)}$$

5) P.T $3n^3 + 2n^2 \in \Theta(n^3)$

$$\rightarrow \text{Given } t(n) = 3n^3 + 2n^2$$

$$g(n) = n^3$$

$$\lim_{n \rightarrow \infty} \frac{t(n)}{g(n)} = \lim_{n \rightarrow \infty} \frac{3n^3 + 2n^2}{n^3}$$

$$= \lim_{n \rightarrow \infty} \frac{n^2(3n+2)}{n^3}$$

$$= \lim_{n \rightarrow \infty} \frac{3n+2}{n} = \lim_{n \rightarrow \infty} 3 + \frac{2}{n}$$

$$= 3 + \frac{2}{\infty} \quad [\because \frac{1}{\infty} = 0]$$

$$= 3 + 0$$

$$= 3 //$$

Hence $t(n) \in \Theta(g(n))$

$$\text{i.e. } \boxed{3n^3 + 2n^2 \in \Theta(n^3)}$$

P.T $100n + 5 \in \Omega(n)$

$$t(n) = 100n + 5 \quad g(n) = n$$

$$2n^2 + 7 \in O(n^3)$$

Let us another example :

P.T $\frac{1}{2}n(n-1) \in \Theta(n^2)$

we have,

$$c_2 g(n) \leq t(n) \leq c_1 g(n) \quad \forall n \geq n_0$$

First let's prove right inequality i.e upper bound

$$t(n) = \frac{1}{2}n(n-1)$$

$$= \frac{1}{2}n^2 - \frac{1}{2}n$$

Let us take $c_1 g(n) = \frac{1}{2}n^2$ [$\because \frac{1}{2}n^2 > \frac{1}{2}n^2 - \frac{1}{2}n$]

$$\therefore \frac{1}{2}n^2 - \frac{1}{2}n \leq \frac{1}{2}n^2 \text{ where } c_1 = \frac{1}{2}$$

if $n=0$, $0 \leq 0$

$n=1$, $0 \leq \frac{1}{2}$

$n=2$, $1 \leq 2$

$\vdots$

Hence $\frac{1}{2}n^2 - \frac{1}{2}n \leq \frac{1}{2}n^2$ For all $n \geq 0$ - (1)

$$\therefore t(n) \leq c_1 g(n)$$

Let's prove left inequality i.e lower bound

$$t(n) = \frac{1}{2}n(n-1) = \frac{1}{2}n^2 - \frac{1}{2}n$$

$\left[\begin{array}{l} \frac{1}{2}n^2(1 - \frac{1}{n}) \\ \frac{1}{2}n^2 \cdot \frac{1}{2} = \frac{1}{4}n^2 \end{array} \right]$ but $1 - \frac{1}{n} \geq \frac{1}{2} \forall n \geq 2$

Let us take $c_2 g(n) = \frac{1}{4}n^2$ [$\because \frac{1}{2}n^2 - \frac{1}{2}n \geq \frac{1}{4}n^2$]

Hence $\frac{1}{2}n^2 - \frac{1}{2}n \geq \frac{1}{4}n^2$ where $c_2 = \frac{1}{4}$

if $n=0$, $0 \geq 0$

$n=1$ $0 \not\geq \frac{1}{4}$

$n=2$ $1 \geq 1$

$n=3$ $3 \geq 2.25$

Hence $\frac{1}{2}n^2 - \frac{1}{2}n \geq \frac{1}{4}n^2 \quad \forall n \geq 2$ - (2)

From (1) & (2) we say that

$$\frac{1}{4}n^2 \leq \frac{1}{2}n^2 - \frac{1}{2}n \leq \frac{1}{2}n^2 \quad \forall n \geq 2 \quad \therefore \boxed{\frac{1}{2}n(n-1) \in \Theta(n^2)}$$

Mathematical Analysis of Nonrecursive Algorithms:-

Following shows the general plan for analyzing non-recursive algorithms -

- * Based on the input size, determine the number of parameters to be considered.

- * Identify the basic operation in the algorithm.

- * Check whether the number of times the basic operation is executed depends only on the size of the input. If the basic operation to be executed depends on some other conditions, then it is necessary to obtain the worst case, best case and average case separately.

- * Obtain the total number of times a basic operation is executed.

- * Simplify using standard formulae, obtain the order of growth.

Example: Calculate the time efficiency of maximum of 'n' elements in a given array.

Algorithm maxelement ($A[0, 1, \dots, n-1]$)

1. // Input : an array of real numbers

2. // Output : Largest element.

3. $\text{max} := A[0]$

4. for $i = 1$ to $n-1$ do

5. if $A[i] > \text{max}$

6. $\text{max} := A[i]$

7. return max.

→ There are two operations in the loop body, the comparison and assignment.

→ We should consider the comparison to be algorithm's basic operation

∴ → Each time Control enters into the loop, the basic operation is executed once.
 → The total number of times the basic operation executed is given by $C(n)$

$$C(n) = \sum_{i=1}^{n-1} 1$$

[∵ For every iteration 1 execution]

$$UB - LB + 1$$

$$= (n-1) - 1 + 1$$

$$C(n) = n-1$$

i.e. $C(n) = \Theta(n-1)$ or

$$T(n) \in \Theta(n-1)$$

Linear Search:- Search for a given key item in a given array.

- Algorithm Linear (key, A[], n)
1. // Searching the position of an element
 2. for $i \leftarrow 0$ to $n-1$ do
 3. if $(A[i] == \text{key})$
 4. return i ;
 5. end for
 6. return -1 ;

Number of times operation executed given by

$$C(n) = \sum_{i=0}^{n-1} 1$$

$$[UB - LB + 1]$$

$$= (n-1) - 0 + 1$$

$$= n - 1 + 1$$

$$C(n) = n$$

∴ $T(n) \in \Theta(n)$

or

$$T(n) \in \Theta(n)$$

→ Best case efficiency for successful search is given by

$$T(n) \in \Omega(1)$$

→ Best case efficiency for unsuccessful search is given by

$$T(n) \in \Omega(n)$$

Matrix multiplication:-

The two matrices a and b can be multiplied and the result can be stored in matrix c for each value of i and j

$$C[i][j] = \sum_{k=0}^{n-1} a[i][k] * b[k][j]$$

Algorithm : multiplication ($a[][]$, $b[][]$)

1. // Input : matrix a of size $n \times n$
matrix b of size $n \times n$
2. // Output : Resultant matrix c of size $n \times n$
3. for $i \leftarrow 0$ to $n-1$ do
4. for $j \leftarrow 0$ to $n-1$ do
5. $C[i, j] \leftarrow 0$;
6. for $k \leftarrow 0$ to $n-1$ do
7. $C[i, j] \leftarrow C[i, j] + a[i, k] * b[k, j]$;
8. end for
9. return $C[i, j]$
10. end for
11. end for.

Time analysis:

$$C(n) = \sum_{i=0}^{n-1} \sum_{j=0}^{n-1} \sum_{k=0}^{n-1} 1$$

$$UB = LB + 1$$

$$= \sum_{i=0}^{n-1} \sum_{j=0}^{n-1} (n - 0 + 1)$$

$$= n \cdot \sum_{i=0}^{n-1} \sum_{j=0}^{n-1} 1$$

$$= n \cdot \sum_{i=0}^{n-1} n$$

$$= n \times n \cdot \sum_{i=0}^{n-1} 1 \rightarrow n \times n \times n = n^3 //$$

$\therefore$ Time complexity to multiply any two matrices of size $n \times n$ is $TCM = \Theta(n^3)$

i.e. $TCM = \Theta(n^3)$

Element Uniqueness Problem:-

Consider an array with 5 elements. For the first time the item $a[0]$ should be compared with $a[1], a[2], a[3]$. Second time, item $a[1]$ should be compared with $a[2], a[3]$ and $a[4]$. & so on, finally $a[3]$ should be compared with $a[4]$

Algorithm: unique ($a[], n$)

1. // Input: An array consisting of 'n' elements
2. // Output: returns 0 if duplicate items are present
returns 1 if the items are unique.

```

3. for i ← 0 to n-2 do
4.     for j ← i+1 to n-1 do
5.         if ( $a[i] = a[j]$ )
6.             return 0;
7.     else
8.         return 1;
9.

```

The time efficiency of this algorithm can be computed as

- $\Rightarrow$ Solve the recurrence relation & obtain the order of growth & express using asymptotic notation.

Factorial of a number:-

$$n! = 1 \cdot 2 \cdot \dots \cdot (n-1) \cdot n = n \cdot (n-1)! \quad \text{for } n \geq 1$$

Formally recursive definition to compute $n!$ is

$$\text{fact}(n) = \begin{cases} 1 & \text{if } n=0 \\ n \cdot (n-1)! & \text{if } n>0 \end{cases}$$

Algorithm $\text{fact}(n)$

1. // compute factorial of a number
2. // Input: A non-negative integer
3. // output: factorial of a given number
4. if $n=0$
5. return 1
6. else return $n * \text{fact}(n-1)$

Analysis:

- ①. The parameter to be considered is n , which is a measure of input size.
- ②. The basic operation is multiplication statement
- ③. The total number of multiplications can be obtained using recurrence relation.

$$T(n) = \begin{cases} 0 & \text{if } n=0 \\ 1 + T(n-1) & \text{otherwise.} \end{cases}$$

let $n=5$,

$$\begin{aligned} T(5) &= 1 + T(5-1) \\ &= 1 + T(4) \\ &= 1 + [1 + T(3)] \\ &= 1 + [1 + [1 + T(2)]] \\ &\quad \text{④} \end{aligned}$$

$$= 1 + [1 + (1 + t(n))]$$

$$= 4 + t(1)$$

$$= 4 + [1 + t(0)]$$

$$\text{But } t(0) = 0$$

$$= 4 + 1$$

$$= \underline{\underline{5}}$$

By using repeated substitution

$$t(n) = 1 + t(n-1)$$

$$= 1 + 1 + t(n-2)$$

$$= 2 + t(n-2)$$

$$= 2 + 1 + t(n-3)$$

$$= 3 + t(n-3)$$

$$= 4 + t(n-4)$$

.....

$$= i + t(n-i)$$

To get the initial condition at $t(0)$. Let us take $i = n$

$$= n + t(n-n)$$

$$= n + t(0)$$

$$\text{But } t(0) = 0$$

$$= \underline{\underline{n}}$$

... The time complexity of factorial of n is $\boxed{t(n) \in \Theta(n)}$

Tower of Hanoi

Algorithm tower_Hanoi(n, A, B, C)

1. if $n = 1$

$A \rightarrow C$

return

2. return tower_Hanoi($n-1, A, C, B$)

$A \rightarrow C$

return tower_Hanoi($n-1, C, A, B$)

Recurrence relation is, (Total movements)

$$t(n) = \begin{cases} 1 & \text{if } n = 1 \\ t(n-1) + 1 + t(n-1) & \text{otherwise (if } n > 1) \end{cases}$$

$$t(n) = 2t(n-1) + 1 \quad [\text{use repeated substitution}]$$

$$= 2[2t(n-1-1) + 1] + 1$$

$$= 2^2 t(n-2) + 2 + 1$$

$$= 2^2 [2t(n-2-1) + 1] + 2 + 1$$

$$= 2^3 t(n-3) + 2^2 + 2 + 1$$

$$= 2^3 [2t(n-3-1) + 1] + 2^2 + 2 + 1$$

$$= 2^4 t(n-4) + 2^3 + 2^2 + 2 + 1$$

⋮

For i th term

$$t(n) = 2^i t(n-i) + 2^{i-1} + 2^{i-2} + \dots + 2^1 + 2^0 \quad \text{--- (1)}$$

We know $2^{i-1} + 2^{i-2} + \dots + 2^1 + 2^0$ is in G.P. (geometric series)

$$S = \frac{a(r^n - 1)}{r - 1}$$

n - no. of terms

a - first no.

r - ~~second~~ common ratio

$$S = \frac{1(2^i - 1)}{2 - 1} = 2^i - 1 \quad \text{--- (2)}$$

Substitute ② in ①

$$t(n) = 2^i t(n-i) + 2^i - 1$$

Assume $i = n-1$

$$t(n) = 2^{n-1} t(n-(n-1)) + 2^{n-1} - 1$$

$$t(n) = 2^{n-1} t(1) + 2^{n-1} - 1$$

$$t(n) = 2 \cdot 2^{n-1} - 1$$

$$t(n) = 2^n - 1$$

$$t(n) = \Theta(2^n)$$

Binary digit:

Algorithm to find the number of binary digits in the binary representation of a positive decimal integer

Algorithm Binary(n)

// Non-recursive algorithm

Input: positive decimal integer n

Output: Number of binary digits in n's binary representation.

count $\leftarrow$ 1

while $n > 1$ do

count $\leftarrow$ count + 1

$n \leftarrow \lfloor n/2 \rfloor$

return count.

Recursive Algorithm:

Algorithm BinRec(n)

Input: A positive decimal integer n

Output: The number of binary digits in n's binary representation

if $n = 1$

return 1

else

return BinRec($\lfloor n/2 \rfloor$) + 1

Time Complexity:

The number of additions made in computing $\text{BinRec}(n/2)$ is $t(n/2)$ plus one more addition is made by the algorithm to increase the returned value.
Recursive call ends when $n=1$ hence no additions made.
So, the no. of additions made is given by

$$t(n) = \begin{cases} 0 & \text{if } n=1 \\ t(n/2) + 1 & \text{if } n>1 \end{cases}$$

Repeated substitution.

$$\begin{aligned} t(n) &= 1 + t(n/2) \\ &= 1 + [1 + t(n/2^2)] \\ &= 1 + 1 + t(n/2^2) \\ &= 2 + t(n/2^2) \\ &= 2 + [1 + t(n/2^3)] \\ &= 3 + t(n/2^3) \\ &= 4 + t(n/2^4) \\ &\vdots \end{aligned}$$

For i th term,

$$t(n) = i + t(n/2^i)$$

to get initial condition $t(1) = 0$,
put $2^i = n$

$$\therefore t(n) = i + t(n/n)$$

$$t(n) = i + t(1)$$

$$t(n) = i \rightarrow \textcircled{1}$$

We assumed $2^i = n$

Take log on both sides

$$\log_2 2^i = \log_2 n$$

$$i \log_2 2 = \log_2 n$$

$$[\because \log_2 2 = 1]$$

$$i = \log_2 n \quad \text{--- (2)}$$

Substitute (2) in (1)

$$T(n) = \log_2 n$$

$$\boxed{T(n) \in \Theta(\log_2 n)}$$

Brute Force design technique

Brute force is a straightforward approach to solving a problem, directly based on the problem statement & definitions of the concepts involved.

Selection Sort:

- Start by scanning the entire list to find the smallest element & exchange it with the first element. Keep the smallest element in the first position in the sorted list.
- Scan the list starting with the second element to find the smallest among last $n-1$ elements & exchange it with the second element adding second smallest element in its final position.
- Repeat the process & after $n-1$ passes the list is sorted.

Algorithm Selection CA $[0, \dots, n-1]$

// Input: An array $A[0, \dots, n-1]$

// Output: Array $A[0, \dots, n-1]$ sorted in ascending

for $i \leftarrow 0$ to $n-2$ do

min $\leftarrow i$

for $j \leftarrow i+1$ to $n-1$ do

if $A[j] < A[\text{min}]$

min $\leftarrow j$

swap $A[i]$ and $A[\text{min}]$

Analysis:

Input Size: Given by the number of elements 'n'

Basic Operation: Comparison.

$$C(n) = \sum_{i=0}^{n-2} \sum_{j=i+1}^{n-1} 1$$

$$= \sum_{i=0}^{n-2} [(n-1) - (i+1) + 1]$$

$$= \sum_{i=0}^{n-2} n-1-i$$

$$= (n-1) + (n-2) + (n-3) + \dots + 3 + 2 + 1$$

$$C(n) = \frac{n \cdot (n-1)}{2}$$

$$= \frac{n^2}{2} - \frac{n}{2}$$

$$C(n) \in O(n^2)$$

Sequential Search: (Linear Search)

This algorithm simply compares successive elements of a given list with a given key element until the match is found or list is exhausted without match.

Algorithm: Sequential ($A[0 \dots n]$, key)

Input: Array of 'n' elements, key

Output: index of an element if key found,
-1 if key not found.

```
for i ← 0 to n do
    if A[i] == key
        return i
```

return -1

Analysis: $O(n)$ for best & average case.

Bubble Sort:-

- This is a sorting problem by comparing adjacent elements of the list & exchange them if they are out of order.
- By doing this repeatedly, it ends up 'bubbling up' the largest element to the last position of the list.
- This can be represented as,

$$A_0 \dots A_j \leftrightarrow A_{j+1} \dots A_{n-i-1} \mid A_{n-i} \leq \dots \leq A_{n-1}$$

Algorithm BubbleSort($A[0 \dots n-1]$)

// Sorts the given array by bubble sort

// input:- An array $A[0 \dots n-1]$ of orderable elements

// output:- Array $A[0 \dots n-1]$ sorted in ascending order.

for $i \leftarrow 0$ to $n-2$ do

for $j \leftarrow 0$ to $n-i-2$ do

if $A[j] > A[j+1]$ then

swap $A[j], A[j+1]$

[End for]

[End for]

Sort the following elements using bubble sort: 14, 7, 10, 5, 2

14, 7, 10, 5, 2

7, 14, 10, 5, 2

7, 10, 14, 5, 2

7, 10, 5, 14, 2

7, 10, 5, 2, 14

pass 1.

7, 10, 5, 2, 14

7, 10, 5, 2, 14

7, 5, 10, 2, 14

7, 5, 2, 10, 14

pass 2

7, 5, 2, 10, 14

5, 7, 2, 10, 14

5, 2, 7, 10, 14

pass 3

5, 2, 7, 10, 14

2, 5, 7, 10, 14

Sorted: 2, 5, 7, 10, 14

Time complexity:

The number of key comparisons for bubble sort version given is same for all arrays of size 'n'.
Basic operation is comparison.

$$C(n) = \sum_{i=0}^{n-2} \sum_{j=0}^{n-i-2} 1$$

$$= \sum_{i=0}^{n-2} (n-i-2+1)$$

$$= \sum_{i=0}^{n-2} (n-i-1)$$

$$= (n-1) + (n-2) + (n-3) + \dots + 3 + 2 + 1$$

$$= 1 + 2 + 3 + \dots + (n-2) + (n-1)$$

$$C(n) = \frac{n(n-1)}{2}$$

$$C(n) = n^2/2 - n/2$$

$$C(n) \in \underline{\underline{\Theta(n^2)}}$$

$$C_{\text{worst}}(n) = \frac{n(n-1)}{2} \in \Theta(n^2)$$

If the elements in sorted order

$$C_{\text{best}}(n) = \Theta(n)$$

Brute - Force String matching

- An brute force algorithm for string matching problem align the pattern against the first 'm' characters of the text & start matching the corresponding pairs of characters from left to right until either all 'm' pairs of the characters match or mismatching pair is encountered.
- In every stage shift the pattern one position to the right and resume character comparisons, starting again with the first character of the pattern & its counterpart in the text.

Algorithm Stringmatch ($T[0, \dots, n-1], P[0, \dots, m-1]$)

// Input: An array $T[0, \dots, n-1]$ of n characters representing a text & an array of $P[0, \dots, m-1]$ of m characters representing a pattern.

// output: Index of first character in the text that starts a matching substring or -1 if fails.

```
for i ← 0 to n-m do
    j ← 0
    while j < m and P[j] = T[i+j] do
        j ← j+1
    [End while]
    if j = m
        return i
[End for]
return -1
```

NOBODY - NOTICED - HIM

NOT

NOT

NOT

NOT

NOT

NOT

NOT

NOT

March found.

Time Complexity

In worst case, the algorithm may have to make all 'm' comparisons before shifting the pattern

$$C(n) = \sum_{i=0}^{n-m} \sum_{j=0}^{m-1}$$

$$= \sum_{i=0}^{n-m} (m - i + 1)$$

$$= m[n - m + 1]$$

$$= mn - m^2 + m$$

$$C(n) = \Theta(mn)$$

Average case efficiency is considerably better than worst

case i.e. $\Theta(n+m) = \underline{\underline{\Theta(n)}}$

Tell a n developer & implement a program to solve bubble sort using brute force technique



Exhaustive Search:

- Exhaustive search is simply a brute-force approach to combinatorial problem
- It suggests generating each & every element of the problem's domain, selecting those of them that satisfy all the constraints & finding the desired element.

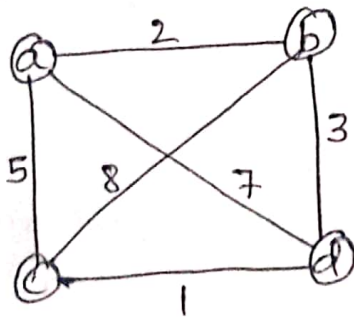
Travelling Salesman Problem:

It is a problem of finding the shortest tour through a given set of 'n' cities that visit each city exactly once before reaching the city where it started.

This problem can be stated as the problem of finding the shortest "Hamiltonian circuit" of the graph.

In TSP, we can get all the tours by generating all the permutations of $n-1$ intermediate cities, compute the tour lengths & find the shortest among them.

Obtain the optimal solution for the following graph by applying exhaustive search



$a \rightarrow b \rightarrow c \rightarrow d \rightarrow a$

$a \rightarrow b \rightarrow d \rightarrow c \rightarrow a$

$a \rightarrow c \rightarrow b \rightarrow d \rightarrow a$

$a \rightarrow c \rightarrow d \rightarrow b \rightarrow a$

$a \rightarrow d \rightarrow b \rightarrow c \rightarrow a$

$a \rightarrow d \rightarrow c \rightarrow b \rightarrow a$

$$l = 2 + 8 + 1 + 7 = 18$$

$$l = 2 + 3 + 1 + 5 = \boxed{11} - \text{Optimal}$$

$$l = 5 + 8 + 3 + 7 = 23$$

$$l = 5 + 1 + 3 + 2 = \boxed{11} - \text{optimal}$$

$$l = 7 + 3 + 8 + 5 = 23$$

$$l = 7 + 1 + 8 + 2 = 18$$

Time Complexity:

The total number of permutations needed will be still $\frac{(n-1)!}{2}$.

It can be given by $O(n!)$ where 'n' is the number of cities.

Knapsack Problem:

This is a problem of choosing a subset of items to fit into a container with a limited capacity.

→ Given 'n' items of known weights $w_1, \dots, w_n$ and values $v_1, \dots, v_n$ & a knapsack capacity W , find the most valuable subset of the items that fit into the knapsack.

Ex: Solve knapsack problem using brute force exhaustive search method for the following problem

$W = 10$		
item 1	7	42
item 2	3	12
item 3	4	40
item 4	5	25

Subset	total weight	Total value
ϕ	ϕ	0
I_1	7	42
I_2	3	12
I_3	4	40
I_4	5	25
I_1, I_2	10	36
I_1, I_3	11	Not feasible
I_1, I_4	12	Not feasible

I_2, I_3	7	52
I_2, I_4	8	37
I_3, I_4	9	65
I_1, I_2, I_3	14	Not feasible
I_1, I_2, I_4	15	Not feasible
I_1, I_3, I_4	16	Not feasible
I_2, I_3, I_4	12	Not feasible
I_1, I_2, I_3, I_4	19	Not feasible.

Hence optimal solution is 65 obtained by placing item 1, item 3 & item 4.

Time Complexity:

Number of subset of an n -element set 2^n , the exhaustive search leads to $\Omega(2^n)$ or $O(2^n)$.

Assignment Problem:

It is an optimization problem that involves assigning a set of jobs to set of people or persons.

- There are ' n ' people who need to be assigned to execute ' n ' jobs, one person per job.

ex: Apply assignment problem for the following set of jobs

	Job1	Job2	Job3
person 1	9	2	7
person 2	6	4	3
person 3	5	8	1

This problem is specified by its cost matrix

$$C = \begin{matrix} & \begin{matrix} J_1 & J_2 & J_3 \end{matrix} \\ \begin{matrix} P_1 \\ P_2 \\ P_3 \end{matrix} & \begin{bmatrix} 9 & 2 & 7 \\ 6 & 4 & 3 \\ 5 & 8 & 1 \end{bmatrix} \end{matrix}$$

Write all possible solutions of assigning jobs to each person

$$\langle 1, 2, 3 \rangle \quad \text{cost} = 9 + 4 + 1 = 14$$

$$\langle 1, 3, 2 \rangle \quad \text{cost} = 9 + 3 + 8 = 20$$

$$\langle 2, 1, 3 \rangle \quad \text{cost} = 2 + 6 + 1 = \boxed{9} - \text{Optimal}$$

$$\langle 2, 3, 1 \rangle \quad \text{cost} = 2 + 3 + 5 = 10$$

$$\langle 3, 1, 2 \rangle \quad \text{cost} = 7 + 6 + 8 = 21$$

$$\langle 3, 2, 1 \rangle \quad \text{cost} = 7 + 4 + 5 = 16$$

Smaller cost among number of assigned task is the optimal solution. Hence optimal solution of g is obtained by assigning job 2 to person 1, job 1 to person 2 & job 3 to person 3.

Time complexity:

The number of permutations to be considered for the general case of assignment problem is $n!$, i.e. for 3 jobs it is 6 permutations. Hence it is $O(n!)$

Note: There is a much more efficient algorithm for this problem called 'Hungarian method' after Hungarian mathematicians König & Egervary

Divide & Conquer

Bif,

Prithviraj
Dept. of CSE

Introduction:

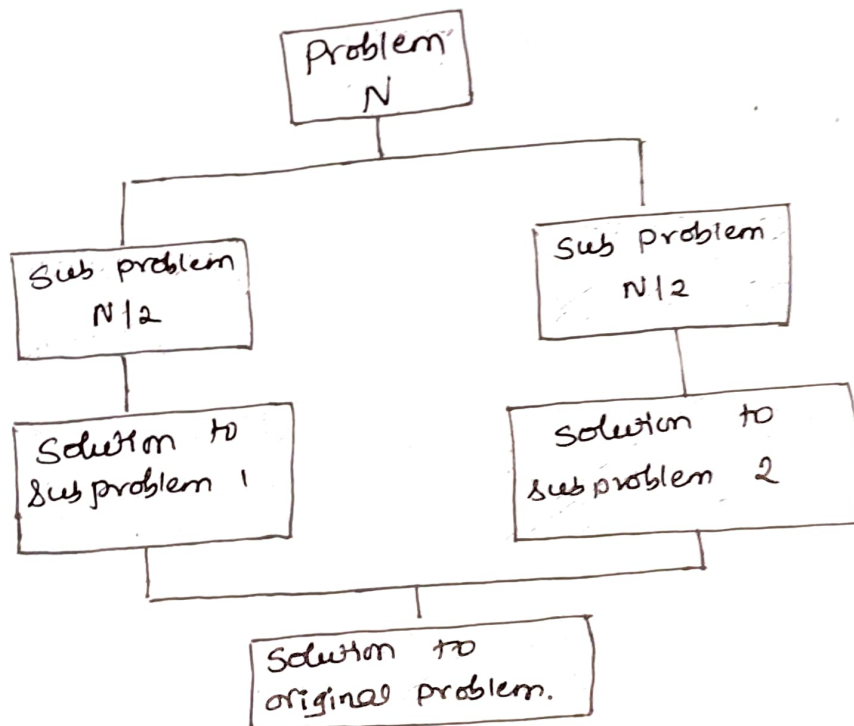
Divide & Conquer is the best known method for solving a problem.

Definition: Divide & Conquer is a top-down technique for designing algorithms that consists of dividing the problem into smaller sub problems & finding the solutions for individual sub problems.

General method:

Divide & Conquer work according to the following general plan.

- i) A problem instance is divided into several smaller instance of the same problem.
- ii) The smaller instances are solved.
- iii) The solutions of the smaller instances are combined to get the solution to the original instance.



Binary Search:-

Binary search is an efficient searching method which can be applied when the elements are in sorted order.

Iterative or non-recursive Binary search:-

Algorithm BinarySearch(a, n, Key)

1. {
2. low = 0, high = n - 1
3. while (low <= high)
4. {
5. mid = (low + high) / 2
6. if (Key < a[mid]) then
7. high = mid - 1;
8. else if (Key > a[mid]) then
9. low = mid + 1;
10. } return mid;
11. return 0;
12. }

example: 10, 20, 30, 40. Apply binary search to find the key element 20

0	1	2	3	
10	20	30	40	50

low = 0

high = 4

n = 5

Key = 20

1st Iteration: high = 5 - 1 = 4
low = 0

(0 ≤ 4)

mid = (0 + 4) / 2 = 2

Key < a[mid] i.e. 20 < 30

high = mid - 1 = 2 - 1 = 1

2nd iteration:

low = 0, high = 1

(0 ≤ 1)

mid = (0+1)/2 = 0

key < a[mid] ⇒ 20 < 10

20 > 10

low = mid + 1 = 0 + 1 = 1

3rd iteration:

low = 1, high = 1

(1 ≤ 1) true

mid = (1+1)/2 = 1

key < a[mid] ⇒ 20 < 20 : false

key > a[mid] ⇒ 20 > 20 : false

return 20.

Recursive Binary Search Algorithm:-

Algorithm BinarySearch(a, l, h, key)

```
{
    if (l = h) then
    {
        if (key = a[l]) then return l;
        else return 0;
    }
    else
    {
        mid = (l + h) / 2;
        if (key = a[mid]) then return mid;
        if (key < a[mid]) then
            return BinarySearch(a, l, mid - 1, key);
        else
            return BinarySearch(a, mid + 1, h, key);
    }
}
```

Master Theorem:

→ Time complexity of divide & conquer technique can be easily computed using master theorem. The recurrence relation can be given by

$$T(n) = a.T(n/b) + f(n)$$

where 'a' and 'b' are the positive constants such that $b > 1$

If $f(n) \in \Theta(n^d)$ with $d \geq 0$ then

$$T(n) = \begin{cases} \Theta(n^d) & \text{if } a < b^d \\ \Theta(n^d \log_2 n) & \text{if } a = b^d \\ \Theta(n^{\log_b a}) & \text{if } a > b^d \end{cases}$$

Hence it can be written as $T(n) = a.T(n/b) + n^d$.

Problem:-

① $T(n) = 2T(n/2) + 1$

master theorem equation is $T(n) = a.T(n/b) + n^d$

$$a = 2, b = 2, d = 0$$

$$a = 2, b^d = 2$$

$$2 > 0$$

$$a > b^d$$

$$\therefore T(n) = \Theta(n^{\log_2 2})$$

$$T(n) = \Theta(n)$$

② $T(n) = 4T(n/2) + n$

m.t. eqn $T(n) = a.T(n/b) + n^d$

$$a = 4, b = 2, d = 1$$

$$a = 4, b^d = 2^1 = 2$$

$$4 > 2$$

$$a > b^d$$

$$T(n) = \Theta(n^{\log_2 4})$$

$$T(n) = \Theta(n^2)$$

$$\textcircled{3} \quad T(n) = kT(n/2) + n^2$$

$$T(n) = aT(n/b) + n^d$$

$$a = k, \quad b = 2, \quad d = 2$$

$$a = k; \quad b^d = 2^2 = 4$$

$$k = 4$$

$$a = b^d$$

$$\therefore T(n) = \Theta(n^d \log n)$$

$$T(n) = \Theta(n^2 \log n)$$

Time Complexity of Binary Search:-

Best case: The best case occurs when the item to be searched is present in the middle of the array. So the total number of comparison required will be 1. Hence the time complexity of binary search in the best case is $\Omega(1)$.

worst case: The worst case occurs when the key element is present at first position or last position in the array. we have $T(n)$ given by following recurrence relation.

$$T(n) = \begin{cases} 1 & \text{if } n=1 \\ 1 + T(n/2) & \text{if } n > 1 \end{cases}$$

where 1 $\rightarrow$ Time required to compare middle element
 $T(n/2) \rightarrow$ Time required to compare upper or lower part of the array.

For worst case we consider the relation

$$T(n) = 1 + T(n/2)$$

$$= 1 + 1 + T(n/2^2)$$

$$= 2 + T(n/2^2)$$

$$T(n/2^2) = 1 + T(n/2^3)$$

$$= 1 + T(n/2^3)$$

$$T(n) = 2 + 1 + T(n/2^3)$$

$$= 3 + T(n/2^3)$$

$$= 4 + T(n/2^4)$$

⋮

In general,

$$T(n) = i + T(n/2^i) \quad \text{--- (1)}$$

To get the initial condition $T(1)$ assume $2^i = n$

$$T(n) = i + T(n/n)$$

$$T(n) = i + T(1)$$

$$T(n) = i + 1 \quad \text{--- (2)}$$

we have $2^i = n$

Apply $\log_2$ on both the sides,

$$i \log_2 2 = \log_2 n$$

$$i = \log_2 n$$

Substitute the value of 'i' in (2)

$$T(n) = \log_2 n + 1$$

$$\boxed{T(n) \in \Theta(\log_2 n)}$$

	Best case	Average case	Worst case
Time complexity	$\Omega(1)$ or $\Theta(1)$	$\Theta(\log_2 n)$	$O(\log_2 n)$ or $\Theta(\log_2 n)$

Advantages:-

- i) Binary search is an optimal searching algorithm using which we can search the desired element very efficiently.
- ii) Simple & efficient searching technique.

Disadvantages:-

This algorithm only applicable if the elements of an array is in sorted order.

Sorting using Divide & Conquer:-

Two sorting techniques

↳ merge sort

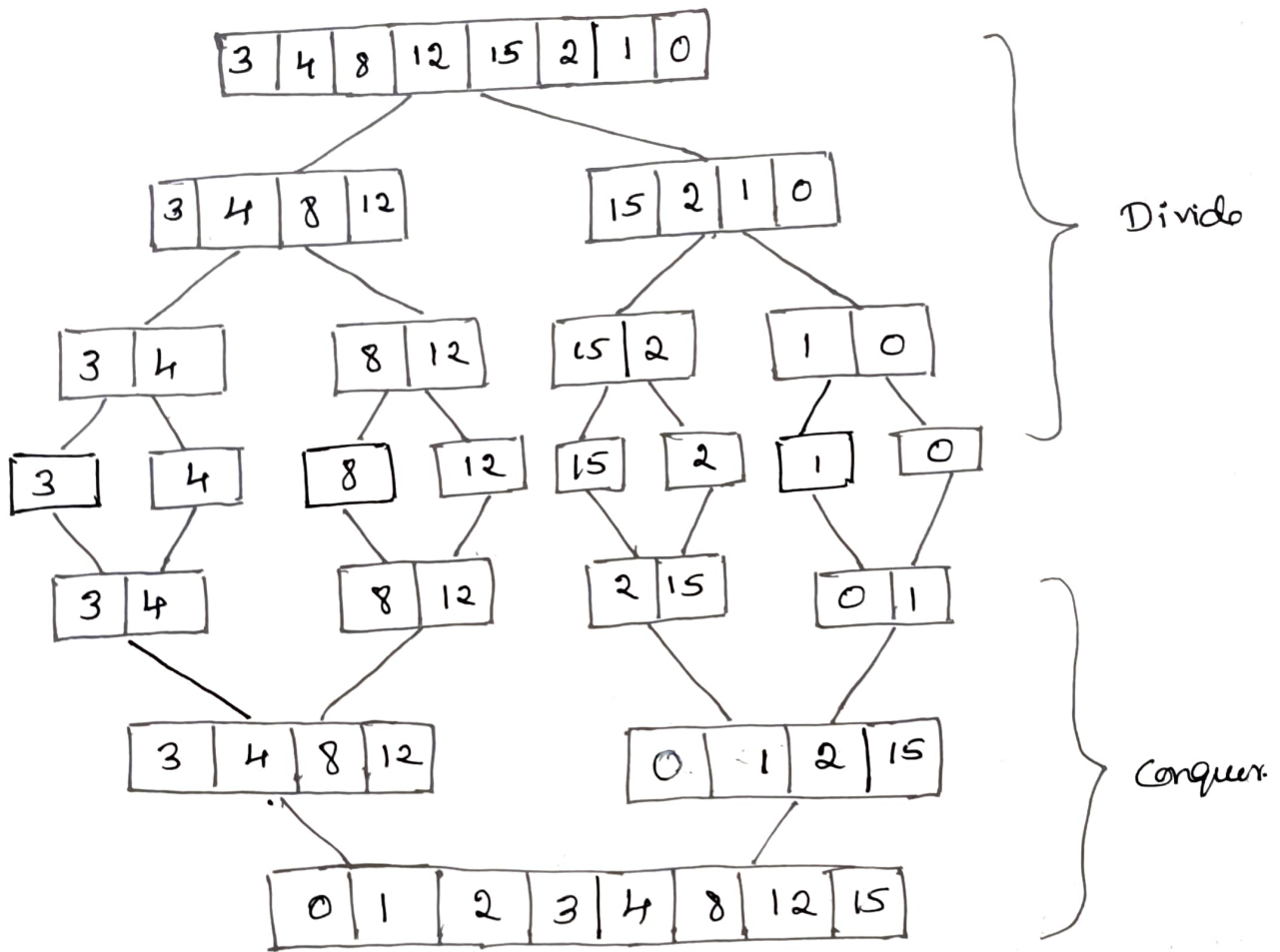
↳ Quick sort

Merge Sort:-

Merge sort is a sorting algorithm that uses divide & conquer strategy. It involves various steps like

- divide - Divide the given array consisting of 'n' elements into two parts of $n/2$ elements each
- conquer - Sort the left part of the array and right part of the array recursively using merge sort
- combine - Combine the sorted left part & right part to get a single sorted array.

Apply merge sort to sort the following elements
3, 4, 8, 12, 15, 2, 10.



Merge Sort Algorithm:-

Algorithm $\text{merge_sort}(a, \text{low}, \text{high})$

if $(\text{low} < \text{high})$

$\text{mid} = (\text{low} + \text{high}) / 2$

$\text{merge_sort}(a, \text{low}, \text{mid})$

$\text{merge_sort}(a, \text{mid} + 1, \text{high})$

$\text{simple_merge}(a, \text{low}, \text{mid}, \text{high})$

end if.

Algorithm $\text{simple_merge}(a, \text{low}, \text{mid}, \text{high})$

$i \leftarrow \text{low}, j \leftarrow \text{mid} + 1, k \leftarrow \text{low}$

while ($i \leq \text{mid}$ and $j \leq \text{high}$)

if ($a[i] < a[j]$) then

$c[k] \leftarrow a[i]$

$i \leftarrow i + 1$

$k \leftarrow k + 1$

else

$c[k] \leftarrow a[j]$

$j \leftarrow j + 1$

$k \leftarrow k + 1$

end if

end while

while ($i \leq \text{mid}$)

$c[k] \leftarrow a[i]$

$k \leftarrow k + 1$

$i \leftarrow i + 1$

end while

while ($j \leq \text{high}$)

$c[k] \leftarrow a[j]$

$k \leftarrow k + 1$

$j \leftarrow j + 1$

end while

for $i = \text{low to high}$

$a[i] \leftarrow c[i]$

end for.

Exercise:- Trace this algorithm to sort the numbers
3, 4, 8, 12, 15, 2, 1, 10

Quicksort

→ This is another sorting algorithm that is based on the divide & conquer approach.

→ Quicksort divides the elements according to their values.

→ It rearranges the elements of a given array $A[0 \dots n-1]$ to achieve its partition, a situation where all the elements before some position 's' are smaller than or equal to $A[s]$ & all the elements after position 's' are greater than or equal to $A[s]$:

$A[0] \dots A[s-1] \ A[s] \ A[s+1] \dots A[n-1]$
all are $\leq A[s]$ All are $\geq A[s]$

Algorithm $QuickSort(A[l \dots h])$

// sorts a subarray by quicksort-

// input: A subarray $A[l \dots h]$ of $A[0 \dots n-1]$ defined by its left & right indices l & h

// output: subarray $A[l \dots h]$ sorted in ~~non~~ increasing order.

if $l < h$

$s \leftarrow \text{partition}(A[l \dots h])$

$QuickSort(A[l \dots s-1])$

$QuickSort(A[s+1 \dots h])$

Algorithm $\text{partition}(A[l \dots h])$

// partitions a subarray by using its first element

// input: A subarray $A[l \dots h]$ of $A[0 \dots n-1]$

// output: A partition of $A[l \dots h]$ with the split position returned as this function's value.

$p \leftarrow A[l]$

$i \leftarrow l$

$j \leftarrow h+1$

while ($i \leq j$)

do repeat

$i \leftarrow i+1$ until $A[i] \geq p$

repeat

$j \leftarrow j-1$ until $A[j] \leq p$

swap ($A[i], A[j]$)

[End while]

swap ($A[i], A[j]$) // undo last swap

swap ($A[i], A[j]$)

return j

Analysis:-

The number of key comparisons made before a partition achieved is $n+1$ if the scanning indices cross over, otherwise n if they collide.

If all the splits happen in the middle of corresponding subarray, we will have best case.

The number of key comparisons in the best case will be satisfy the following recurrence (Best case)

$$T(n) = \begin{cases} 0 & n=1 \\ T(n/2) + T(n/2) + n & \text{for } n > 1 \end{cases}$$

↳ Time taken to partition 'n' elements

$$T(n) = 2T(n/2) + n$$

This is of the form $T(n) = aT(n/b) + f(n)$

where $a=2, b=2, d=1$

$$a: 2 = 2^1$$

$$2 = 2,$$

$$\therefore a = b^d$$

i.e

which relates to $\Theta(n \log n)$

$$T(n) \in \Theta(n \log n)$$

Worst-case Analysis:-

In General if the array has 'n' elements, after partitioning n-1 elements will be there towards right & 0 elements will be there towards left of the key element.

The recurrence relation is

$$T(n) = \begin{cases} 0 & \text{if } n=1 \\ T(0) + T(n-1) + n \end{cases}$$

$T(0)$ - No elements towards left / right

$T(n-1)$ - Time required to sort remaining n-1 elements.

n - Time required to partition the array.

$$T(n) = T(0) + T(n-1) + n$$

$$T(n) = T(n-1) + n$$

$$= [T(n-1-1) + (n-1)] + n$$

$$= T(n-2) + (n-1) + n$$

$$= [T(n-2-1) + (n-2)] + (n-2) + n$$

$$= T(n-3) + (n-2) + (n-1) + n$$

⋮

$$= T(n-n) + 1 + 2 + \dots + (n-2) + (n-1) + n$$

$$= 0 + 1 + 2 + \dots + (n-2) + (n-1) + n$$

$$= \frac{n(n+1)}{2} \leq n^2 \text{ for } n \geq 0$$

$$= \frac{n^2}{2} + \frac{n}{2} \in \Theta(n^2)$$

∴ Time complexity of quick sort in worst case is

$$T(n) = \Theta(n^2)$$

Average case time efficiency:

→ The key element may be placed at any arbitrary position in the array ranging from 0 to $n-1$

Time to sort left-part = $T(k)$

Time to sort right part = $T(n-1-k)$

The time complexity of quick sort in average case is

$$T(n) = O(n \log_2 n)$$

Advantages:-

- 1) Quick sort algorithm is in-place since it uses only a small auxiliary stack.
- 2) Quick sort ~~require~~ time complexity of $O(n \log_2 n)$ in best case & average case to sort 'n' items.
- 3) It has extremely short inner loops.

Disadvantages:-

- 1) Not Stable
- 2) Since algorithm is recursive it is difficult for the beginners to trace & grasp.
- 3) Time complexity of quick sort is quadratic i.e. $O(n^2)$ time in worst case
- 4) Simple mistake in the implementation can go unnoticed & algorithm may not work.

Apply Quick Sort to sort the following elements in

Ascending order: 5, 3, 1, 9, 8, 2, 4, 7

	0	1	2	3	4	5	6	7	h
1	5	3	1	9	8	2	4	7	

(P) i

5	3	1	9	8	2	4	7
---	---	---	---	---	---	---	---

5	3	1	4	8	2	9	7
---	---	---	---	---	---	---	---

5	3	1	4	8	2	9	7
---	---	---	---	---	---	---	---

5	3	1	4	2	8	9	7
---	---	---	---	---	---	---	---

5	3	1	4	2	8	9	7
---	---	---	---	---	---	---	---

2	3	1	4	5	8	9	7
---	---	---	---	---	---	---	---

Take 2nd part

Take first partition

	0	1	2	3	h
2	2	3	1	4	

(P) i

2	3	1	4
---	---	---	---

2	1	3	4
---	---	---	---

2	1	3	4
---	---	---	---

1	2	3	4
---	---	---	---

1	2	3	4
---	---	---	---

3	4
---	---

3	4
---	---

	0	1	2	3	h
5	8	9	7		

(P) i

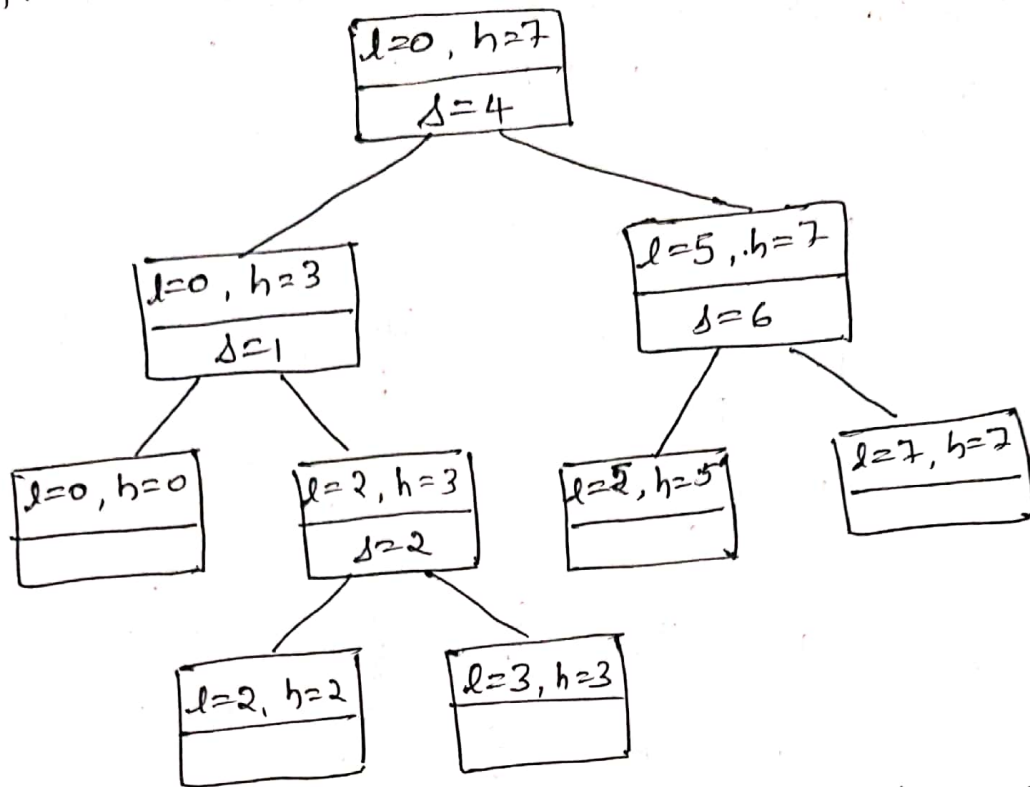
8	7	9
---	---	---

8	7	9
---	---	---

7	8	9
---	---	---

7	8	9
---	---	---

Recursive Tree



Apply Quicksort to sort 'QUICKSORT' & draw recursive call tree.